<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>KASE ERP – Skill Development</title>

<style>

*{box-sizing:border-box}

:root{--blue:#004578;--accent:#0078d4;--bg:#f3f2f1;--line:#d2d0ce;--muted:#605e5c}

body{margin:0;height:100vh;display:flex;flex-direction:column;overflow:hidden;font:12px "Segoe UI",Arial,sans-serif;color:#201f1e;background:var(--bg)}

header{height:50px;flex-shrink:0;background:var(--blue);color:white;display:flex;align-items:center;justify-content:space-between;padding:0 12px;gap:10px}

.brand,.right{display:flex;align-items:center;gap:9px}

.logo-img{width:40px;height:40px;object-fit:contain;background:white;padding:2px;border-radius:2px}

header input{width:210px;padding:7px;border:0;font:inherit}

header button{background:none;color:white;border:1px solid #8fb8dc;padding:6px 10px;cursor:pointer;font:inherit}

#app{display:flex;flex:1;min-height:0}

aside{width:245px;flex-shrink:0;border-right:1px solid var(--line);overflow:auto;padding:5px 0;background:#f3f2f1}

.navhead{padding:11px 12px 5px;font-weight:bold;color:var(--muted)}

.navitem{display:block;width:100%;text-align:left;background:none;border:0;border-left:4px solid transparent;padding:9px 12px 9px 13px;font:inherit;cursor:pointer;color:#201f1e;min-height:36px}

.navitem:hover{background:#e8e6e4}

.navitem.active{background:white;border-left-color:var(--accent);font-weight:600;color:var(--blue)}

main{flex:1;min-width:0;display:flex;flex-direction:column;overflow:hidden}

.toolbar{background:white;padding:9px 14px;border-bottom:1px solid var(--line);display:flex;align-items:center;gap:10px;min-height:57px}

.crumb{font-size:11px;color:var(--muted)}

h1{font-size:18px;font-weight:600;margin:3px 0 0}

```
.ribbon{background:#faf9f8;border-bottom:1px solid var(--line);padding:0 9px}

.ribbon-title{display:inline-block;color:var(--accent);font-weight:bold;padding:7px 4px 4px;border-bottom:3px solid var(--accent)}

.groups{display:flex;flex-wrap:wrap}

.group{padding:3px 8px;border-right:1px solid #ddd}

.rbtn{display:inline-flex;flex-direction:column;align-items:center;justify-content:center;min-width:65px;min-height:43px;background:none;border:1px solid transparent;padding:4px;cursor:pointer;font:inherit}

.rbtn i{font-style:normal;font-size:16px;color:var(--accent)}

.rbtn:hover{background:#e8e6e4}

.group small{display:block;text-align:center;color:var(--muted);font-size:10px}

.content{flex:1;overflow:auto;padding:13px 14px}

footer{background:#e1dfdd;color:var(--muted);padding:4px 12px;font-size:11px;display:flex;justify-content:space-between;flex-shrink:0}

.card{background:white;border:1px solid var(--line);padding:12px;margin-bottom:12px}

.card h3{margin:0 0 9px;font-size:13px;color:var(--blue)}

.cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:9px;margin-bottom:12px}

.metric{background:white;border:1px solid var(--line);border-top:4px solid var(--accent);padding:10px;min-width:0}

.clickable{cursor:pointer}

.clickable:hover{background:#f1f8fd}

.metric span{color:var(--muted)}

.metric strong{display:block;font-size:22px;margin-top:6px;color:var(--blue);overflow-wrap:anywhere}

.layout{display:grid;grid-template-columns:minmax(0,1fr) 230px;gap:12px;align-items:start}

.tablewrap{overflow:auto;background:white;border:1px solid var(--line);margin-bottom:10px}

table{width:100%;border-collapse:collapse}

th{background:#f3f2f1;color:var(--muted);text-align:left;padding:8px;border-bottom:1px solid var(--line);white-space:nowrap}

td{padding:8px;border-bottom:1px solid #edebe9;vertical-align:top}

tbody tr{cursor:pointer}

tbody tr:hover td{background:#f1f8fd}
```

.badge{display:inline-block;padding:3px 7px;background:#fff4ce;color:#835b00;white-space:nowrap}

.badge.green{background:#dff6dd;color:#0b6a0b}

.badge.red{background:#fde7e9;color:#a4262c}

.badge.blue{background:#deecf9;color:#004578}

.filters{display:flex;gap:8px;align-items:center;flex-wrap:wrap;margin-bottom:10px}

.filters input,.filters select{padding:7px;border:1px solid #8a8886;background:white;font:inherit}

.btn{font:inherit;padding:6px 11px;border:1px solid #8a8886;background:white;cursor:pointer;margin:2px}

.btn.primary{background:var(--accent);border-color:var(--accent);color:white}

.btn:disabled{opacity:.45;cursor:not-allowed}

.sidebox{background:white;border:1px solid var(--line);padding:11px}

.sidebox h4{margin:5px 0 8px;color:var(--blue);border-bottom:1px solid var(--line);padding-bottom:5px}

.history{padding:7px 0;border-bottom:1px solid #edebe9}

.muted{color:var(--muted)}

details.section{background:white;border:1px solid var(--line);margin-bottom:9px}

details.section summary{padding:9px 12px;font-weight:bold;color:var(--blue);cursor:pointer;background:#faf9f8}

.formgrid{display:grid;grid-template-columns:repeat(auto-fit,minmax(190px,1fr));gap:12px;padding:12px}

.field label{display:block;color:var(--muted);font-size:11px;margin-bottom:4px}

.field input,.field select,.field textarea{width:100%;font:inherit;padding:7px;border:1px solid #8a8886;background:white}

.field textarea{min-height:65px}

.empty{padding:22px;text-align:center;color:var(--muted)}

.toast{position:fixed;right:15px;bottom:30px;background:#dff6dd;color:#0b6a0b;border:1px solid #107c10;padding:10px 14px;display:none;z-index:100;max-width:400px}

.overlay{position:fixed;inset:0;background:#0006;display:none;align-items:center;justify-content:center;z-index:60}

.overlay.open{display:flex}

.modal{background:white;border:1px solid #888;width:92%;max-width:680px;max-height:90vh;overflow:auto}

.modalhead{background:var(--blue);color:white;padding:11px 14px;font-weight:600}

```
.modalbody{padding:14px}

.modalactions{text-align:right;margin-top:12px}

.tile{cursor:pointer}

.tile:hover{background:#f1f8fd}


/* Full-screen login without the application behind it */

body.login-mode{background:#f3f2f1}

body.login-mode>header,

body.login-mode>#app,

body.login-mode>footer{display:none!important}

body.login-mode>.overlay{display:flex!important;background:#f3f2f1;z-index:200}

body.login-mode .modal{max-width:520px;width:calc(100% - 32px);border:1px solid #c8c6c4;box-shadow:0 6px 24px #00000018;max-height:calc(100vh - 32px)}

body.login-mode .modalhead{padding:13px 16px}

body.login-mode .modalbody{padding:18px 16px}

.login-intro{margin-top:4px;margin-bottom:16px;color:#333}

.login-hint{margin-top:14px;color:var(--muted);line-height:1.5}

.login-actions{display:flex;justify-content:flex-end;margin-top:18px}


@media(max-width:850px){aside{width:205px}.layout{grid-template-columns:1fr}header input{width:140px}}

@media(max-width:600px){aside{display:none}.content{padding:8px}header input{width:110px}}

</style>

</head>

<body>

<header>

  <div class="brand">

    <img class="logo-img" src="https://drive.google.com/thumbnail?id=1ZPt4G_Eb2YvHfqR7PO3zg4dulEMZB0ym&amp;sz=w200" alt="KASE Logo" onerror="this.style.display='none';document.getElementById('logoFallback').style.display='inline-block'">
```

```
   <strong id="logoFallback"
style="display:none;background:white;color:#004578;padding:8px">KASE</strong>
   <strong>KASE ERP</strong><span>|</span><span>Skill Development</span>
  </div>
  <div class="right">
   <input id="globalSearch" placeholder="Search modules or records...">
   <span id="currentUser">Not signed in</span>
   <button onclick="logout()">Sign out</button>
  </div>
</header>

<div id="app">
  <aside id="nav"></aside>
  <main id="main"></main>
</div>

<footer>
  <span>Kerala Academy for Skills Excellence · Skill Development Module</span>
  <span id="dateLabel"></span>
</footer>

<div class="toast" id="toast"></div>
<div class="overlay" id="modalOverlay">
  <div class="modal">
   <div class="modalhead" id="modalTitle">Action</div>
   <div class="modalbody" id="modalBody"></div>
  </div>
</div>

<script>
"use strict";
```

```
const $=id=>document.getElementById(id);
const today=()=>new Date().toLocaleDateString("en-GB");
const isoToday=()=>new Date().toISOString().slice(0,10);
const money=n=>"Rs. "+Number(n||0).toLocaleString("en-IN",{maximumFractionDigits:2});

let role=null;
let activeModule="home";
let detailId=null;
let selectedSkillSchemeId=null;
let selectedProposalId=null;
let selectedSkillBatchId=null;
let pendingActions=[];

const roles={
 manager:{label:"Manager (Skill Development)",permissions:["all"]},
 assistant:{label:"Assistant Manager (Skill Development)",permissions:["review","configure","schedule","verify","operations","score","batchReview","level2","trainerReview","candidateMobilize","attendance","assessorMap","stateBudget","districtBudget"]},
 accreditation:{label:"Executive (Accreditation)",permissions:["scrutiny","prepare","schedule","level1","candidateReview","batchReview","trainerReview"]},
 coe:{label:"Executive (CoE)",permissions:["operations","verify","prepare","candidateMobilize","attendance","dsdcOps"]}
};

const demoAccounts={
 manager:{password:"Demo@123",role:"manager"},
 assistant:{password:"Demo@123",role:"assistant"},
 "executive.accreditation":{password:"Demo@123",role:"accreditation"},
 "executive.coe":{password:"Demo@123",role:"coe"}
};
```

```
/* Navigation: text only, no icons */
const navGroups=[
 {name:"Overview",items:[["home","Dashboard"]]},
 {name:"Skill Development",items:[
  ["skillDashboard","Skill Development Dashboard"],
  ["skillSchemes","Centrally Sponsored Schemes"],
  ["stateSchemes","State Schemes"],
  ["skillProposals","TSP Proposals & Targets"],
  ["skillTrainers","Trainer & ToT Verification"],
  ["skillCandidates","Candidate Registry & Mobilisation"],
  ["skillBatches","Batch Requests & Approval"],
  ["skillAttendance","Attendance & Dropouts"],
  ["skillAssessment","Assessment & Certification"],
  ["skillPlacement","OJT & Placements"],
  ["skillMilestones","Milestone Payments / FICO"],
  ["dsdc","DSDC Operations & Budgets"],
  ["stateLedgers","State Budget Ledger Mapping"]
 ]},
 {name:"Reports",items:[["reports","Reports & Audit"]]}
];

const moduleNames=Object.fromEntries(navGroups.flatMap(g=>g.items.map(i=>[i[0],i[1]])));

const db={
 skillSchemes:[
  {id:"CSS-001",name:"PMKVY",stream:"Central",fundSource:"Centrally Sponsored Scheme",guidelines:"PMKVY-guidelines.pdf",template:"TSP-proposal-template.pdf",applyLink:true,status:"Published",ledger:"",history:[]},
  {id:"CSS-002",name:"SANKALP",stream:"Central",fundSource:"Centrally Sponsored Scheme",guidelines:"SANKALP-guidelines.pdf",template:"TSP-proposal-template.pdf",applyLink:true,status:"Published",ledger:"",history:[]},
```

{id:"STS-001",name:"State Plan Skill Development Scheme",stream:"State",fundSource:"Plan Fund",guidelines:"State-scheme-guidelines.pdf",template:"Proposal-template.pdf",applyLink:true,status:"Published",ledger:"PLAN-2026",history:[]},

{id:"STS-002",name:"SC Development Fund Training",stream:"State",fundSource:"Other Department Fund",guidelines:"SC-development-guidelines.pdf",template:"Proposal-template.pdf",applyLink:true,status:"Published",ledger:"SC-DEV-2026",history:[]}

],

skillProposals:[

{id:"PROP-001",scheme:"CSS-001",tsp:"Keltron",category:"Government",accreditationId:"Not Required",background:"Government TSP",district:"Statewide",proposedTarget:150,target:150,level1:"Pending",level2:"Pending",status:"Submitted",workAward:"",stream:"Central",ledger:"",history:[]},

{id:"PROP-002",scheme:"CSS-001",tsp:"Malabar Digital Skills Foundation",category:"Accredited",accreditationId:"ACC-2026-011",background:"Accreditation profile retrieved",district:"Malappuram",proposedTarget:100,target:0,level1:"Verified",level2:"Pending",status:"Level 2 Pending",workAward:"",stream:"Central",ledger:"",history:[]},

{id:"PROP-003",scheme:"STS-001",tsp:"Kerala Technical Learning Centre",category:"Accredited",accreditationId:"ACC-2025-008",background:"Accreditation profile retrieved",district:"Kollam",proposedTarget:80,target:80,level1:"Verified",level2:"Reviewed",status:"Approved",workAward:"WA-2026-003",stream:"State",ledger:"PLAN-2026",history:[]}

],

trainers:[

{id:"TRN-001",tsp:"Keltron",name:"Anil Kumar",trade:"IT Support",totNumber:"TOT-IT-1044",totValidUntil:"2027-06-30",status:"Verified",history:[]},

{id:"TRN-002",tsp:"Malabar Digital Skills Foundation",name:"Meera Nair",trade:"Digital Skills",totNumber:"TOT-DS-2210",totValidUntil:"2027-01-31",status:"Verified",history:[]},

{id:"TRN-003",tsp:"Kerala Technical Learning Centre",name:"Rajesh P",trade:"Construction",totNumber:"TOT-CT-0032",totValidUntil:"2026-08-31",status:"Expired",history:[]}

],

candidates:[

{id:"CAND-2026-001",name:"Sample Candidate 001",district:"Thiruvananthapuram",qualification:"Plus Two",contact:"*****1234",aadhaarMasked:"XXXX-XXXX-1234",status:"Mobilised",scheme:"CSS-001",mobilizedProposal:"PROP-001",history:[]},

{id:"CAND-2026-002",name:"Sample Candidate 002",district:"Kollam",qualification:"Degree",contact:"*****2456",aadhaarMasked:"XXXX-XXXX-2456",status:"Mobilised",scheme:"CSS-001",mobilizedProposal:"PROP-001",history:[]},

{id:"CAND-2026-003",name:"Sample Candidate 003",district:"Malappuram",qualification:"Plus Two",contact:"*****9911",aadhaarMasked:"XXXX-XXXX-9911",status:"Registered",scheme:"CSS-001",mobilizedProposal:"",history:[]},

{id:"CAND-2026-004",name:"Sample Candidate 004",district:"Kollam",qualification:"ITI",contact:"*****4700",aadhaarMasked:"XXXX-XXXX-4700",status:"Mobilised",scheme:"STS-001",mobilizedProposal:"PROP-003",history:[]}

],

skillBatches:[

{id:"SKB-001",proposal:"PROP-003",name:"KTL-KLM-01",district:"Kollam",trainer:"TRN-002",candidateIds:["CAND-2026-004"],capacity:25,status:"Active",stream:"State",budgetId:"DSDC-KLM",start:"2026-10-01",end:"2027-01-31",history:[]},

{id:"SKB-002",proposal:"PROP-001",name:"KEL-IT-01",district:"Thiruvananthapuram",trainer:"TRN-001",candidateIds:["CAND-2026-001","CAND-2026-002"],capacity:30,status:"Pending Executive Verification",stream:"Central",budgetId:"",start:"2026-11-01",end:"2027-02-28",history:[]}

],

attendanceLogs:[

{id:"ATT-SK-001",batch:"SKB-001",candidate:"CAND-2026-004",date:"2026-10-19",status:"Present"},

{id:"ATT-SK-002",batch:"SKB-001",candidate:"CAND-2026-004",date:"2026-10-20",status:"Present"}

],

assessments:[],

certificates:[],

postTraining:[],

stateLedgers:[

{id:"PLAN-2026",name:"Plan Fund 2026",scheme:"STS-001",allocated:5000000,utilized:0},

{id:"SC-DEV-2026",name:"SC Development Fund",scheme:"STS-002",allocated:2500000,utilized:0}

],

dsdcBudgets:[

{id:"DSDC-KLM",district:"Kollam",source:"Plan Fund 2026",ledger:"PLAN-2026",allocated:1000000,utilized:0,officer:"District Skill Coordinator – Kollam"},

{id:"DSDC-MPM",district:"Malappuram",source:"Plan Fund 2026",ledger:"PLAN-2026",allocated:1200000,utilized:0,officer:"District Skill Coordinator – Malappuram"},

{id:"DSDC-TVPM",district:"Thiruvananthapuram",source:"SC Development Fund",ledger:"SC-DEV-2026",allocated:800000,utilized:0,officer:"District Skill Coordinator – Thiruvananthapuram"}

],

```
 ficoQueue:[],
 reports:[]
};

const settings={attendanceThreshold:80,passMark:50};

function esc(v){return String(v??"").replace(/[&<>"']/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#39;"}[c]));}
function badge(s){
 let c="";
if(/Active|Published|Approved|Qualified|Valid|Resolved|Verified|Completed|Archived|Placed|Running|Present|Certificate Generated|Achieved/.test(s||""))c="green";
 if(/Rejected|Failed|Expired|Breach|Dropout|Invalid/.test(s||""))c="red";
 return `<span class="badge ${c}">${esc(s||"—")}</span>`;
}
function can(p){return !!role&&(roles[role].permissions.includes("all")||roles[role].permissions.includes(p));}
function toast(msg){
 $("toast").textContent="✓ "+msg;
 $("toast").style.display="block";
 clearTimeout(window.toastTimer);
 window.toastTimer=setTimeout(()=>$("toast").style.display="none",3500);
}
function openModal(title,body){
 $("modalTitle").textContent=title;
 $("modalBody").innerHTML=body;
 $("modalOverlay").classList.add("open");
}
function closeModal(){$("modalOverlay").classList.remove("open");}
function log(r,msg){
 r.history=r.history||[];
```

```
 r.history.unshift(today()+" · "+(roles[role]?.label||"System")+": "+msg);
}
function find(m,id){return (db[m]||[]).find(x=>x.id===id);}
function scheme(id){return find("skillSchemes",id);}
function prop(id){return find("skillProposals",id);}
function trainer(id){return find("trainers",id);}
function candidate(id){return find("candidates",id);}
function batch(id){return find("skillBatches",id);}
function schemeName(id){return scheme(id)?.name||id||"—";}
function proposalName(id){return prop(id)?.tsp||id||"—";}
function validTrainer(t){return !!t&&!!t.totValidUntil&&new Date(t.totValidUntil)>=new
Date(isoToday());}
function countByStatus(rows,status){return rows.filter(x=>x.status===status).length;}
function proposalCandidates(id){return db.candidates.filter(c=>c.mobilizedProposal===id);}
function proposalMobilized(id){return
proposalCandidates(id).filter(c=>c.status!=="Dropout").length;}
function activeCandidates(b){return
(b.candidateIds||[]).map(candidate).filter(c=>c&&c.status!=="Dropout");}
function certFor(id){return db.certificates.find(c=>c.candidate===id);}
function postFor(id){return db.postTraining.find(c=>c.candidate===id);}
function attPercentage(cid,bid){
 const
rows=db.attendanceLogs.filter(a=>a.candidate===cid&&a.batch===bid&&a.status!=="Dropout");
 return rows.length?Math.round(rows.filter(a=>a.status==="Present").length/rows.length*100):0;
}
function fieldDisplay(label,value){return `<div
class="field"><label>${esc(label)}</label><div>${esc(value??"—")}</div></div>`;}
function metricCard(label,value,action=""){
 return `<div class="metric ${action?"clickable":""}"
${action?`onclick="${action}"`:""}><span>${esc(label)}</span><strong>${esc(value)}</strong></div>`
;
}
function tableHtml(headers,rows,getValues,onClick){
```

```
 return `<div
class="tablewrap"><table><thead><tr>${headers.map(h=>`<th>${esc(h)}</th>`).join("")}</tr></thea
d><tbody>${rows.length?rows.map(r=>`<tr
${onClick?`onclick="${onClick(r)}"`:""}>${getValues(r).map((v,i)=>`<td>${i===headers.length-
1&&headers[i]==="Status"?badge(v):esc(v??"—")}</td>`).join("")}</tr>`).join(""):`<tr><td
colspan="${headers.length}">No records available.</td></tr>`}</tbody></table></div>`;

}


/* Text-only navigation; there are no icon elements or icon mappings. */

function navBuild(){

 $("nav").innerHTML=navGroups.map(g=>

  `<div class="navhead">${g.name}</div>`+

  g.items.map(([id,label])=>

   `<button class="navitem ${activeModule===id?"active":""}"
onclick="openModule('${id}')">${label}</button>`

  ).join("")

 ).join("");

}

function resetSelections(){

 detailId=null;

 selectedSkillSchemeId=null;

 selectedProposalId=null;

 selectedSkillBatchId=null;

}

function openModule(id){activeModule=id;resetSelections();render();}


/* Full-screen login state; no application backdrop is visible. */

function loginDialog(){

 document.body.classList.add("login-mode");

 $("modalOverlay").classList.add("open");

 openModal("KASE ERP Login",`

  <p class="login-intro">Sign in using your assigned KASE account.</p>
```

```
 <div class="field"><label for="loginUser">User ID</label><input id="loginUser"
autocomplete="username" placeholder="Enter user ID" autofocus></div>

 <div class="field"><label for="loginPass">Password</label><input id="loginPass" type="password"
autocomplete="current-password" placeholder="Enter password"
onkeydown="if(event.key==='Enter')login()"></div>

 <div class="login-hint">Demo accounts: <b>manager</b>, <b>assistant</b>,
<b>executive.accreditation</b>, <b>executive.coe</b><br>Demo password:
<b>Demo@123</b></div>

 <div class="login-actions"><button class="btn primary" onclick="login()">Sign in</button></div>`);

 setTimeout(()=>$("loginUser")?.focus(),30);

}

function login(){

 const u=($("loginUser")?.value||"").trim().toLowerCase();

 const p=$("loginPass")?.value||"";

 const a=demoAccounts[u];

 if(!a||a.password!==p){toast("Invalid demo credentials");return;}

 role=a.role;

 $("currentUser").textContent=roles[role].label;

 document.body.classList.remove("login-mode");

 closeModal();

 render();

 toast("Signed in as "+roles[role].label);

}

function logout(){

 role=null;

 activeModule="home";

 resetSelections();

 $("currentUser").textContent="Not signed in";

 document.body.classList.add("login-mode");

 render();

 loginDialog();

}

function page(title,crumb,body,actions=""){
```

```
 return `<div class="toolbar"><div><div
class="crumb">${esc(crumb)}</div><h1>${esc(title)}</h1></div><div style="margin-
left:auto">${esc(roles[role]?.label||"")}</div></div><div class="ribbon"><div class="ribbon-
title">HOME</div>${actions?`<div class="groups"><div
class="group">${actions}<small>Actions</small></div></div>`:""}</div><div
class="content">${body}</div>`;
}

function render(){
 navBuild();
 if(!role){$("main").innerHTML="";return;}
 const routes={
  home:renderHome,
  skillDashboard:renderSkillDashboard,
  skillSchemes:()=>renderSchemes("Central"),
  stateSchemes:()=>renderSchemes("State"),
  skillProposals:renderProposals,
  skillTrainers:renderTrainers,
  skillCandidates:renderCandidates,
  skillBatches:renderSkillBatches,
  skillAttendance:renderSkillAttendance,
  skillAssessment:renderAssessment,
  skillPlacement:renderSkillPlacement,
  skillMilestones:renderMilestones,
  dsdc:renderDSDC,
  stateLedgers:renderStateLedgers,
  reports:renderReports
 };
 const fn=routes[activeModule];
 $("main").innerHTML=fn?fn():'<div class="content"><div class="card">Module
unavailable.</div></div>';
}
```

```
function exportCSV(name,rows){
 const csv="\uFEFF"+rows.map(row=>row.map(v=>'"'+String(v??"").replace(/"/g,'""')+'"').join(",")).join("\r\n");
 const url=URL.createObjectURL(new Blob([csv],{type:"text/csv;charset=utf-8"}));
 const a=document.createElement("a");
 a.href=url;a.download=name;a.click();
 URL.revokeObjectURL(url);
 toast("CSV export generated");
}
function exportModule(m){
 const rows=db[m]||[];
 if(!rows.length){toast("No records to export");return;}
 const keys=[...new Set(rows.flatMap(r=>Object.keys(r).filter(k=>!["history","scores","candidateIds","marks"].includes(k)))];
 exportCSV(m+"-export.csv",[keys,...rows.map(r=>keys.map(k=>Array.isArray(r[k])?r[k].join("; "):r[k]??""))]);
}

function renderHome(){
 return page("ERP Dashboard","KASE ERP › Overview",`
 <div class="card"><h3>Management Overview</h3><div class="cards">
 ${metricCard("Schemes",db.skillSchemes.length,"openModule('skillSchemes')")}
 ${metricCard("TSP Proposals",db.skillProposals.length,"openModule('skillProposals')")}
 ${metricCard("Candidates",db.candidates.length,"openModule('skillCandidates')")}
 ${metricCard("Skill Batches",db.skillBatches.length,"openModule('skillBatches')")}
 ${metricCard("District Budgets",db.dsdcBudgets.length,"openModule('dsdc')")}
 ${metricCard("FICO Requests",db.ficoQueue.length,"openModule('skillMilestones')")}
 </div></div>
 <div class="cards">
 ${metricCard("Skill Development","Open module","openModule('skillDashboard')")}
 ${metricCard("Candidate Mobilisation","Open module","openModule('skillCandidates')")}
```

```
 ${metricCard("Batch Approval","Open module","openModule('skillBatches')")}

 </div>`);

}

function renderSkillDashboard(){

 const pending=db.skillProposals.filter(p=>!["Approved","Rejected"].includes(p.status));

 return page("Skill Development Dashboard","KASE ERP › Skill Development",`

 <div class="card"><h3>Training Operations Overview</h3><div class="cards">

 ${metricCard("Centrally Sponsored
Schemes",db.skillSchemes.filter(s=>s.stream==="Central").length,"openModule('skillSchemes')")}

 ${metricCard("State
Schemes",db.skillSchemes.filter(s=>s.stream==="State").length,"openModule('stateSchemes')")}

 ${metricCard("Pending Proposals",pending.length,"openModule('skillProposals')")}

 ${metricCard("Active
Batches",countByStatus(db.skillBatches,"Active"),"openModule('skillBatches')")}

 ${metricCard("Certificates",db.certificates.length,"openModule('skillAssessment')")}


${metricCard("Placements",db.postTraining.filter(p=>p.placementStatus==="Placed").length,"open
Module('skillPlacement')")}

 </div></div>

 <div class="cards">

 <div class="metric clickable" onclick="openModule('skillSchemes')"><span>Stream 1</span><strong
style="font-size:17px">Centrally Sponsored Schemes</strong><p>PMKVY, SANKALP and other
central schemes.</p></div>

 <div class="metric clickable" onclick="openModule('stateSchemes')"><span>Stream
2</span><strong style="font-size:17px">State Schemes</strong><p>Plan Fund and other
departmental funds.</p></div>

 <div class="metric clickable" onclick="openModule('dsdc')"><span>Stream 3</span><strong
style="font-size:17px">DSDC Operations</strong><p>District-level execution and
budgets.</p></div>

 </div>

 <div class="card"><h3>Pending Proposals</h3>


${tableHtml(["Proposal","TSP","Scheme","Status"],pending,p=>[p.id,p.tsp,schemeName(p.scheme),p
.status],p=>`openProposal('${p.id}')`)}

 </div>`);

}
```

```
function renderSchemes(stream){
 const rows=db.skillSchemes.filter(s=>s.stream===stream);
 return page(stream==="Central"?"Centrally Sponsored Schemes":"State Schemes","KASE ERP › Skill Development › Schemes",`
 <div class="card"><h3>Scheme Register</h3>
 <p class="muted">Approved scheme information and application links are published through the website. TSP submissions enter ERP through the portal.</p>
 ${tableHtml(["Scheme ID","Scheme","Funding Source","Guidelines","Apply Link","Status"],rows,s=>[s.id,s.name,s.fundSource,s.guidelines,s.applyLink?"Active":"Inactive",s.status],s=>`openScheme('${s.id}')`)}
 </div>
 ${stream==="State"?`<div class="card"><h3>State Budget Mapping</h3>
 <button class="btn primary" onclick="openModule('stateLedgers')">Open Ledger Mapping</button>
 <button class="btn" onclick="openModule('dsdc')">Open DSDC Budgets</button></div>`:""}`);
}
function openScheme(id){
 const s=find("skillSchemes",id);
 openModal("Scheme Details – "+s.name,`
 <div class="formgrid">
 ${fieldDisplay("Scheme ID",s.id)}${fieldDisplay("Scheme",s.name)}${fieldDisplay("Stream",s.stream)}
 ${fieldDisplay("Funding Source",s.fundSource)}${fieldDisplay("Guidelines PDF",s.guidelines)}
 ${fieldDisplay("Proposal Template",s.template)}${fieldDisplay("Application Link",s.applyLink?"Active":"Inactive")}
 ${fieldDisplay("Publication",s.status)}${fieldDisplay("FICO Ledger",s.ledger||"Not mapped")}
 </div>
 ${can("all")?`<button class="btn primary" onclick="toggleSchemePublish('${id}')">Toggle Publication</button><button class="btn" onclick="toggleApplyLink('${id}')">Toggle Apply Link</button>`:""}
 <button class="btn" onclick="openTSPIntake('${id}')">Preview Public Application</button>
 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);
}
function toggleSchemePublish(id){
```

```
 if(!can("all")){toast("Manager approval required");return;}
 const s=find("skillSchemes",id);
 s.status=s.status==="Published"?"Draft":"Published";
 log(s,"Publication changed");
 closeModal();render();toast("Publication updated");
}
function toggleApplyLink(id){
 if(!can("all")){toast("Manager approval required");return;}
 const s=find("skillSchemes",id);
 s.applyLink=!s.applyLink;
 log(s,"Application link changed");
 closeModal();render();toast("Application link updated");
}
function openTSPIntake(preselected=""){
 openModal("Public Website – TSP Scheme Application",`
 <p>Simulated external website application process.</p>
 <div class="field"><label>Scheme</label><select
id="intakeScheme">${db.skillSchemes.filter(s=>s.status==="Published"&&s.applyLink).map(s=>`<opt
ion value="${s.id}"
${s.id===preselected?"selected":""}>${esc(s.name)}</option>`).join("")}</select></div>
 <div class="field"><label>TSP Name</label><input id="intakeTSP"></div>
 <div class="field"><label>TSP Category</label><select id="intakeCategory"
onchange="updateIntakeCategory()"><option>Government</option><option>Accredited</option><
option>Non-Accredited</option></select></div>
 <div class="field" id="accreditationField"><label>Accreditation ID</label><input
id="intakeAcc"></div>
 <div class="field"><label>District</label><input id="intakeDistrict"></div>
 <div class="field"><label>Proposed Candidate Target</label><input id="intakeTarget"
type="number" min="1" value="100"></div>
 <p class="muted">Government TSPs bypass accreditation checks. Non-accredited applicants must
complete accreditation first.</p>
 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button
class="btn primary" onclick="submitTSPIntake()">Continue</button></div>`);
 updateIntakeCategory();
```

```
}
function updateIntakeCategory(){
 const f=$("accreditationField");
 if(f)f.style.display=$("intakeCategory").value==="Accredited"?"block":"none";
}
function submitTSPIntake(){
 const s=find("skillSchemes",$("intakeScheme").value);
 const tsp=$("intakeTSP").value.trim();
 const cat=$("intakeCategory").value;
 const acc=$("intakeAcc").value.trim();
 const district=$("intakeDistrict").value.trim();
 const target=Number($("intakeTarget").value);
 if(!s||!tsp||!target||target<1){toast("Complete the mandatory fields");return;}
 if(cat==="Non-Accredited"){
  closeModal();
  openModal("Accreditation Required",`<p>${esc(tsp)} must complete the accreditation process before applying for this scheme.</p><div class="modalactions"><button class="btn primary" onclick="closeModal()">Close</button></div>`);
  return;
 }
 if(cat==="Accredited"&&!acc){toast("Accreditation ID is required");return;}
 if(cat==="Accredited"&&!/^ACC-/i.test(acc)){toast("Demonstration Accreditation ID not recognised");return;}
 const id="PROP-"+Date.now().toString().slice(-6);
 db.skillProposals.unshift({id,scheme:s.id,tsp,category:cat,accreditationId:cat==="Government"?"Not Required":acc,background:cat==="Government"?"Government TSP":"Accreditation profile retrieved",district:district||"Statewide",proposedTarget:target,target:0,level1:"Pending",level2:"Pending",status:"Submitted",workAward:"",stream:s.stream,ledger:s.ledger||"",history:["Received through public website demo"]});
 closeModal();openModule("skillProposals");toast("Portal proposal received in demo");
}

/* TSP PROPOSAL REVIEW */
```

```
function renderProposals(){
 const rows=db.skillProposals;
 return page("TSP Proposals & Target Allocation","KASE ERP › Skill Development › Proposals",`
 <div class="card"><h3>Submitted Proposals</h3>
 <p class="muted">TSP submissions arrive through the public website. ERP staff perform proposal review, vetting and target allocation.</p>
 ${tableHtml(["Proposal","TSP","Scheme","Category","Proposed Target","Allocated Target","Level 1","Level
2","Status"],rows,p=>[p.id,p.tsp,schemeName(p.scheme),p.category,p.proposedTarget,p.target,p.level1,p.level2,p.status],p=>`openProposal('${p.id}')`)}
 </div>`);
}
function openProposal(id){
 const p=find("skillProposals",id);
 selectedProposalId=id;
 const actions=[];
 if(["Submitted","Return for
Correction"].includes(p.status)&&(can("level1")||can("all")))actions.push(`<button class="btn primary" onclick="proposalAction('${id}','verify1')">Level 1 Verify</button><button class="btn" onclick="proposalAction('${id}','return')">Return for Correction</button>`);
 if(p.status==="Level 2 Pending"&&(can("level2")||can("all")))actions.push(`<button class="btn primary" onclick="proposalAction('${id}','verify2')">Level 2 Review</button>`);
 if(p.status==="Pending Manager Approval"&&can("all"))actions.push(`<button class="btn primary" onclick="proposalAction('${id}','approve')">Approve & Allocate Target</button><button class="btn" onclick="proposalAction('${id}','reject')">Reject</button>`);
 openModal("Proposal – "+p.id,`
 <div class="formgrid">

${fieldDisplay("TSP",p.tsp)}${fieldDisplay("Scheme",schemeName(p.scheme))}${fieldDisplay("Category",p.category)}
 ${fieldDisplay("Accreditation ID",p.accreditationId)}${fieldDisplay("District",p.district)}
 ${fieldDisplay("Proposed Target",p.proposedTarget)}${fieldDisplay("Allocated Target",p.target)}
 ${fieldDisplay("Mobilised",proposalMobilized(id))}${fieldDisplay("Level 1",p.level1)}
 ${fieldDisplay("Level 2",p.level2)}${fieldDisplay("Status",p.status)}
 ${fieldDisplay("Work Award",p.workAward||"Not generated")}
```

```
 </div>
 ${actions.join("")}
 <h3>History</h3>${(p.history||[]).map(h=>`<div class="history">${esc(h)}</div>`).join("")}
 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);
}
function proposalAction(id,act){
 const p=find("skillProposals",id);
 if(act==="verify1"){
  if(!can("level1")&&!can("all"))return;
  p.level1="Verified";p.status="Level 2 Pending";log(p,"Level 1 verification completed");
 }
 if(act==="return"){
  if(!can("level1")&&!can("all"))return;
  p.level1="Return for Correction";p.status="Return for Correction";log(p,"Returned for correction");
 }
 if(act==="verify2"){
  if(!can("level2")&&!can("all"))return;
  p.level2="Reviewed";p.status="Pending Manager Approval";log(p,"Level 2 review completed");
 }
 if(act==="approve"){
  if(!can("all"))return;
  if(p.level1!=="Verified"||p.level2!=="Reviewed"){toast("Complete both review levels first");return;}
  p.status="Approved";p.target=Number(p.proposedTarget||0);p.workAward="WA-
"+Date.now().toString().slice(-6);
  if(p.stream==="State")p.ledger=find("skillSchemes",p.scheme)?.ledger||"";
  log(p,"Approved; target allocated and Work Award generated");
 }
 if(act==="reject"){
  if(!can("all"))return;
  p.status="Rejected";log(p,"Proposal rejected");
 }
```

```
 closeModal();render();toast("Proposal workflow updated");
}

/* TRAINER AND ToT VERIFICATION */
function renderTrainers(){
 return page("Trainer & ToT Verification","KASE ERP › Skill Development › Trainers",`
 <div class="card"><h3>Trainer Register</h3>
 <p class="muted">Batch allocation requires a valid Training of Trainers certificate.</p>
 ${tableHtml(["ID","TSP","Trainer","Trade","ToT Certificate","Valid Until","Status"],db.trainers,t=>[t.id,t.tsp,t.name,t.trade,t.totNumber,t.totValidUntil,t.status],t=>`openTrainer('${t.id}')`)}
 </div>`);
}
function openTrainer(id){
 const t=find("trainers",id);
 openModal("Trainer Details – "+t.name,`
 <div class="formgrid">${fieldDisplay("TSP",t.tsp)}${fieldDisplay("Trainer",t.name)}
 ${fieldDisplay("Trade",t.trade)}${fieldDisplay("ToT Reference",t.totNumber)}
 ${fieldDisplay("Valid Until",t.totValidUntil)}${fieldDisplay("Validation",validTrainer(t)?"Current":"Expired or invalid")}</div>
 ${can("trainerReview")||can("all")?`<button class="btn primary" onclick="verifyTrainer('${id}')">Recheck Validity</button>`:""}
 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);
}
function verifyTrainer(id){
 const t=find("trainers",id);t.status=validTrainer(t)?"Verified":"Expired";log(t,"ToT validity checked");
 closeModal();render();toast("Trainer certificate checked");
}

/* CANDIDATE REGISTRY AND MOBILISATION */
function renderCandidates(){
 return page("Candidate Registry & Mobilisation","KASE ERP › Skill Development › Candidates",`
```

```
<div class="card"><h3>Public Website Candidate Register</h3>

<p class="muted">Public registration originates through the website. Aadhaar is masked in this demonstration.</p>

<button class="btn primary" onclick="openCandidatePortal()">Preview Candidate Registration</button>

<button class="btn" onclick="openMobiliseCandidate()">Mobilise Candidate</button>

<div class="cards">${metricCard("Registered",countByStatus(db.candidates,"Registered"))}${metricCard("Mobilised",countByStatus(db.candidates,"Mobilised"))}${metricCard("Dropouts",countByStatus(db.candidates,"Dropout"))}</div>

${tableHtml(["Candidate ID","Candidate","District","Qualification","Scheme","Status","Proposal"],db.candidates,c=>[c.id,c.name,c.district,c.qualification,schemeName(c.scheme),c.status,c.mobilizedProposal||"—"],c=>`openCandidate('${c.id}')`)}

</div>`);

}

function openCandidatePortal(){

openModal("Public Website – Candidate Registration",`

<p>Demonstration of the public registration form.</p>

<div class="field"><label>Candidate Name</label><input id="canName"></div>

<div class="field"><label>District</label><input id="canDistrict"></div>

<div class="field"><label>Qualification</label><select id="canQual"><option>SSLC</option><option>Plus Two</option><option>ITI</option><option>Diploma</option><option>Degree</option><option>Postgraduate</option></select></div>

<div class="field"><label>Contact (masked demo value)</label><input id="canContact" placeholder="*****1234"></div>

<div class="field"><label>Last four Aadhaar digits (demo only)</label><input id="canAadhaar" maxlength="4"></div>

<div class="field"><label>Scheme of Interest</label><select id="canScheme">${db.skillSchemes.filter(s=>s.status==="Published").map(s=>`<option value="${s.id}">${esc(s.name)}</option>`).join("")}</select></div>

<div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="submitCandidate()">Register</button></div>`);

}

function submitCandidate(){
```

```
 const
name=$("canName").value.trim(),district=$("canDistrict").value.trim(),a=$("canAadhaar").value.trim
();

 if(!name||!district){toast("Name and district are required");return;}

 if(a&&!/^\d{4}$/.test(a)){toast("Use four demo digits only");return;}

 const id="CAND-"+new Date().getFullYear()+"-"+Date.now().toString().slice(-6);


db.candidates.unshift({id,name,district,qualification:$("canQual").value,contact:$("canContact").val
ue||"Not supplied",aadhaarMasked:a?"XXXX-XXXX-"+a:"Not
supplied",status:"Registered",scheme:$("canScheme").value,mobilizedProposal:"",history:[]});

 closeModal();render();toast("Candidate registered: "+id);

}

function openCandidate(id){

 const c=find("candidates",id);

 openModal("Candidate Profile – "+c.name,`

 <div class="formgrid">${fieldDisplay("ID",c.id)}${fieldDisplay("Name",c.name)}

 ${fieldDisplay("District",c.district)}${fieldDisplay("Qualification",c.qualification)}

 ${fieldDisplay("Contact",c.contact)}${fieldDisplay("Masked Aadhaar",c.aadhaarMasked)}

 ${fieldDisplay("Scheme",schemeName(c.scheme))}${fieldDisplay("Status",c.status)}

 ${fieldDisplay("Proposal",c.mobilizedProposal||"Not assigned")}</div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);

}

function openMobiliseCandidate(){

 const ps=db.skillProposals.filter(p=>p.status==="Approved"&&p.target>proposalMobilized(p.id));

 const cs=db.candidates.filter(c=>c.status==="Registered"&&!c.mobilizedProposal);

 if(!ps.length||!cs.length){toast("No eligible candidates or available approved targets");return;}

 openModal("TSP Portal – Candidate Mobilisation",`

 <div class="field"><label>Approved Proposal</label><select id="mobProp">${ps.map(p=>`<option
value="${p.id}">${esc(p.id)} – ${esc(p.tsp)} (remaining ${p.target-
proposalMobilized(p.id)})</option>`).join("")}</select></div>

 <div class="field"><label>Candidate</label><select id="mobCandidate">${cs.map(c=>`<option
value="${c.id}">${esc(c.id)} – ${esc(c.name)}</option>`).join("")}</select></div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button
class="btn primary" onclick="mobiliseCandidate()">Mobilise</button></div>`);
```

```
}
function mobiliseCandidate(){
 const p=find("skillProposals",$("mobProp").value),c=find("candidates",$("mobCandidate").value);
 if(!p||!c||p.status!=="Approved"){toast("Select an approved proposal and registered candidate");return;}
 if(p.target-proposalMobilized(p.id)<=0){toast("Target reached");return;}
 if(c.mobilizedProposal||c.status!=="Registered"){toast("Candidate is not eligible");return;}
 c.mobilizedProposal=p.id;c.status="Mobilised";log(c,"Mobilised against "+p.id);
 closeModal();render();toast("Candidate mobilised");
}

/* BATCH REQUESTS AND APPROVAL */
function renderSkillBatches(){
 return page("Batch Requests & Approval","KASE ERP › Skill Development › Batches",`
 <div class="card"><h3>Batch Formation Requests</h3>
 <p class="muted">Requests originate through the TSP/DSDC portal. Attendance is enabled only after Manager approval.</p>
 <button class="btn primary" onclick="openBatchRequest()">Preview Batch Request Portal</button>

${tableHtml(["Batch","TSP","Scheme","District","Trainer","Candidates","Status"],db.skillBatches,b=>[b.id,proposalName(b.proposal),schemeName(prop(b.proposal)?.scheme),b.district,find("trainers",b.trainer)?.name||b.trainer,b.candidateIds.length,b.status],b=>`openBatch('${b.id}')`)}
 </div>`);
}
function openBatchRequest(){
 const ps=db.skillProposals.filter(p=>p.status==="Approved"),ts=db.trainers.filter(validTrainer);
 if(!ps.length||!ts.length){toast("Approved proposal and valid trainer are required");return;}
 openModal("TSP Portal – Batch Formation Request",`
 <div class="field"><label>Approved Proposal</label><select id="brProposal" onchange="updateBatchOptions()">${ps.map(p=>`<option value="${p.id}">${esc(p.id)} – ${esc(p.tsp)}</option>`).join("")}</select></div>
 <div class="field"><label>Batch Name</label><input id="brName"></div>
 <div class="field"><label>Trainer with valid ToT</label><select id="brTrainer">${ts.map(t=>`<option value="${t.id}">${esc(t.name)} – ${esc(t.trade)}</option>`).join("")}</select></div>
```

```
 <div class="field"><label>District</label><input id="brDistrict"></div>

 <div class="field"><label>Capacity</label><input id="brCapacity" type="number" min="1" value="30"></div>

 <div class="field"><label>Start Date</label><input id="brStart" type="date"></div>

 <div class="field"><label>End Date</label><input id="brEnd" type="date"></div>

 <div id="brCandidates">${candidateChecks(ps[0].id)}</div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="submitBatchRequest()">Submit Request</button></div>`);

}

function candidateChecks(pid){

 return `<div class="field"><label>Mobilised candidates</label>${db.candidates.filter(c=>c.status==="Mobilised"&&c.mobilizedProposal===pid&&!db.skillBatches.some(b=>b.candidateIds.includes(c.id)&&b.status!=="Rejected")).map(c=>`<label><input type="checkbox" name="brCandidate" value="${c.id}" style="width:auto"> ${esc(c.id)} – ${esc(c.name)}</label>`).join("")||'<p class="muted">No eligible candidates.</p>'}</div>`;

}

function updateBatchOptions(){$("brCandidates").innerHTML=candidateChecks($("brProposal").value);}

function submitBatchRequest(){

 const p=find("skillProposals",$("brProposal").value),t=find("trainers",$("brTrainer").value);

 const ids=[...document.querySelectorAll('input[name="brCandidate"]:checked')].map(x=>x.value),cap=Number($("brCapacity").value);

 if(!p||!t||!validTrainer(t)){toast("A valid ToT trainer is required");return;}

 if(!ids.length||!cap||ids.length>cap){toast("Select candidates within batch capacity");return;}

 if(ids.some(id=>find("candidates",id)?.mobilizedProposal!==p.id)){toast("Candidate does not match proposal");return;}

 db.skillBatches.unshift({id:"SKB-"+Date.now().toString().slice(-6),proposal:p.id,name:$("brName").value.trim()||"BATCH-"+Date.now().toString().slice(-4),district:$("brDistrict").value.trim()||p.district,trainer:t.id,candidateIds:ids,capacity:cap,status:"Pending Executive Verification",stream:p.stream,budgetId:"",start:$("brStart").value,end:$("brEnd").value,history:[]});

 closeModal();render();toast("Batch request submitted");

}

function openBatch(id){

 const b=find("skillBatches",id),t=find("trainers",b.trainer),actions=[];
```

```
 if(b.status==="Pending Executive Verification"&&(can("batchReview")||can("all")))actions.push(`<button class="btn primary" onclick="batchAction('${id}','verify')">Executive – Verify</button>`);

 if(b.status==="Pending Manager Approval"&&can("all"))actions.push(`<button class="btn primary" onclick="batchAction('${id}','approve')">Manager – Approve & Activate</button>`);

 if(b.status==="Active"&&(can("attendance")||can("all")))actions.push(`<button class="btn primary" onclick="openAttendanceEntry('${id}')">Record Attendance</button><button class="btn" onclick="completeBatch('${id}')">Mark Training Completed</button>`);

 openModal("Batch – "+b.name,`

 <div class="formgrid">${fieldDisplay("Batch ID",b.id)}${fieldDisplay("TSP",proposalName(b.proposal))}

${fieldDisplay("Scheme",schemeName(prop(b.proposal)?.scheme))}${fieldDisplay("District",b.district)}

 ${fieldDisplay("Trainer",t?.name)}${fieldDisplay("ToT Status",validTrainer(t)?"Valid":"Expired / Invalid")}

 ${fieldDisplay("Candidates",b.candidateIds.length)}${fieldDisplay("Capacity",b.capacity)}

 ${fieldDisplay("Status",b.status)}</div>

 <h3>Candidate Roster</h3>${tableHtml(["Candidate ID","Name","Status"],b.candidateIds.map(findCandidate).filter(Boolean),c=>[c.id,c.name,c.status])}

 ${actions.join("")}

 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);

}

function findCandidate(id){return find("candidates",id);}

function batchAction(id,act){

 const b=find("skillBatches",id),t=find("trainers",b.trainer);

 if(act==="verify"){

  if(!can("batchReview")&&!can("all"))return;

  if(!validTrainer(t)){toast("Trainer ToT expired or invalid");return;}

  if(!b.candidateIds.length||b.candidateIds.length>b.capacity){toast("Batch capacity check failed");return;}

  b.status="Pending Manager Approval";log(b,"Executive verified batch");

 }

 if(act==="approve"){

  if(!can("all"))return;
```

```
 if(!validTrainer(t)){toast("Cannot activate batch with invalid ToT");return;}
 b.status="Active";log(b,"Manager approved batch; attendance enabled");
 }
 closeModal();render();toast("Batch workflow updated");
}
function completeBatch(id){
 if(!can("all")){toast("Manager permission required");return;}
 const b=find("skillBatches",id);if(!b||b.status!=="Active")return;
 b.status="Training Completed";log(b,"Training completion recorded");
 closeModal();render();toast("Batch completed");
}

/* ATTENDANCE AND DROPOUT TRACKING */
function renderSkillAttendance(){
 return page("Attendance & Dropout Tracking","KASE ERP › Skill Development › Attendance",`
 <div class="card"><h3>Batch Attendance Register</h3>
 <p class="muted">Only Active batches accept attendance.</p>
 ${tableHtml(["Batch","Provider","District","Active Candidates","Status","Latest Entry"],db.skillBatches,b=>[b.name,proposalName(b.proposal),b.district,activeCandidates(b).length,b.status,db.attendanceLogs.filter(a=>a.batch===b.id).map(a=>a.date).sort().pop()||"None"],b=>`openAttendanceBatch('${b.id}')`)}
 <p class="muted">Illustrative attendance threshold: ${settings.attendanceThreshold}%. Confirm scheme-specific rules before operational use.</p>
 </div>`);
}
function openAttendanceBatch(id){
 const b=find("skillBatches",id),logs=db.attendanceLogs.filter(a=>a.batch===id),roster=activeCandidates(b);
 openModal("Attendance – "+b.name,`
 ${b.status==="Active"&&(can("attendance")||can("all"))?`<button class="btn primary" onclick="openAttendanceEntry('${id}')">Record Daily Attendance</button>`:"<p class='muted'>Attendance entry is available only for Active batches.</p>"}
```

```
 <h3>Attendance History</h3>${tableHtml(["Date","Candidate","Status"],logs,a=>[a.date,find("candidates",a.candidate)?.name||a.candidate,a.status])}

 <h3>Candidate Summary</h3>

 ${tableHtml(["Candidate","Present","Sessions","Attendance %","Status"],roster,c=>{const ls=logs.filter(x=>x.candidate===c.id);return[c.name,ls.filter(x=>x.status==="Present").length,ls.length,attPercentage(c.id,id)+"%",c.status]})}

 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);

}

function openAttendanceEntry(id){

 const b=find("skillBatches",id);

 if(!b||b.status!=="Active"){toast("Only Active batches accept attendance");return;}

 if(!can("attendance")&&!can("all")){toast("Your role cannot record attendance");return;}

 openModal("TSP Portal – Daily Attendance",`

 <p>Batch: <b>${esc(b.name)}</b>. Mark Present, Absent or Dropout.</p>

 <div class="field"><label>Date</label><input id="attDate" type="date" value="${isoToday()}"></div>

 <div class="tablewrap"><table><thead><tr><th>Candidate</th><th>Status</th></tr></thead><tbody>

 ${activeCandidates(b).map(c=>`<tr><td>${esc(c.id)} – ${esc(c.name)}</td><td><select id="att_${c.id}"><option>Present</option><option>Absent</option><option>Dropout</option></select></td></tr>`).join("")}

 </tbody></table></div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="saveAttendance('${id}')">Save Attendance</button></div>`);

}

function saveAttendance(id){

 const b=find("skillBatches",id),date=$("attDate").value;

 if(!date){toast("Attendance date required");return;}

 activeCandidates(b).forEach(c=>{

  const status=$("att_"+c.id)?.value;if(!status)return;

  const old=db.attendanceLogs.find(x=>x.batch===id&&x.candidate===c.id&&x.date===date);

  if(old)old.status=status;
```

```
 else db.attendanceLogs.push({id:"ATT-"+Date.now().toString().slice(-6),batch:id,candidate:c.id,date,status});
 if(status==="Dropout"){c.status="Dropout";log(c,"Removed from active roster");}
 });
 log(b,"Attendance recorded for "+date);
 closeModal();render();toast("Attendance saved");
}

/* ASSESSMENT AND CERTIFICATION */
function renderAssessment(){
 return page("Assessment & Certification","KASE ERP › Skill Development › Assessment",`
 <div class="card"><h3>Assessment Sessions</h3>
 <p class="muted">Only completed batches are eligible for assessment mapping.</p>
 <button class="btn primary" onclick="mapAssessorDialog()">Map Assessment Agency</button>
 ${tableHtml(["Session","Batch","Provider","Agency","Eligible Candidates","Status"],db.assessments,a=>[a.id,find("skillBatches",a.batch)?.name,proposalName(find("skillProposals",find("skillBatches",a.batch)?.proposal)?.id),a.agency,a.candidateIds.length,a.status],a=>`openAssessment('${a.id}')`)}
 </div>
 <div class="card"><h3>Certificates</h3>${tableHtml(["Certificate","Candidate","Batch","Marks","Issued","Status"],db.certificates,c=>[c.id,find("candidates",c.candidate)?.name,find("skillBatches",c.batch)?.name,c.marks,c.issued,c.status])}</div>`);
}
function mapAssessorDialog(){
 const rows=db.skillBatches.filter(b=>b.status==="Training Completed");
 if(!rows.length){toast("Complete a training batch before mapping an assessor");return;}
 openModal("Map External Assessment Agency",`
 <div class="field"><label>Completed Batch</label><select id="assBatch">${rows.map(b=>`<option value="${b.id}">${esc(b.name)}</option>`).join("")}</select></div>
 <div class="field"><label>Assessment Agency</label><input id="assAgency"></div>
 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="saveAssessmentMapping()">Map Agency</button></div>`);
}
```

```
function saveAssessmentMapping(){
 const b=find("skillBatches",$("assBatch").value),agency=$("assAgency").value.trim();
 if(!b||!agency){toast("Batch and agency required");return;}
 const eligible=activeCandidates(b).filter(c=>attPercentage(c.id,b.id)>=settings.attendanceThreshold);
 if(!eligible.length){toast("No candidate meets attendance threshold");return;}
 if(db.assessments.some(a=>a.batch===b.id)){toast("Assessment already mapped to this batch");return;}
 db.assessments.unshift({id:"ASM-"+Date.now().toString().slice(-6),batch:b.id,agency,candidateIds:eligible.map(c=>c.id),marks:[],status:"Agency Mapped",history:[]});
 closeModal();render();toast("Assessment agency mapped");
}
function openAssessment(id){
 const a=find("assessments",id);
 openModal("Assessment – "+a.id,`
 <div class="formgrid">${fieldDisplay("Batch",find("skillBatches",a.batch)?.name)}${fieldDisplay("Agency",a.agency)}${fieldDisplay("Eligible Candidates",a.candidateIds.length)}${fieldDisplay("Status",a.status)}</div>
 ${a.status!=="Marks Submitted"?`<button class="btn primary" onclick="enterAssessmentMarks('${id}')">Enter Marks</button>`:""}

<h3>Results</h3>${tableHtml(["Candidate","Marks","Outcome"],a.marks,m=>[find("candidates",m.candidate)?.name,m.marks,m.outcome])}
 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);
}
function enterAssessmentMarks(id){
 const a=find("assessments",id);
 openModal("External Assessment Marks",`
 <p>Illustrative pass mark: ${settings.passMark}%. Certificates are generated for eligible passing candidates.</p>
 <div class="tablewrap"><table><thead><tr><th>Candidate</th><th>Attendance</th><th>Marks / 100</th></tr></thead><tbody>

${a.candidateIds.map(cid=>`<tr><td>${esc(find("candidates",cid)?.name)}</td><td>${attPercentage(
```

```
cid,a.batch)}%</td><td><input id="mark_${cid}" type="number" min="0" max="100" value="60" style="width:80px"></td></tr>`).join("")}

 </tbody></table></div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="saveAssessmentMarks('${id}')">Save Marks & Generate Certificates</button></div>`);

}

function saveAssessmentMarks(id){

 const a=find("assessments",id);a.marks=[];

 for(const cid of a.candidateIds){

  const marks=Number($("mark_"+cid).value);

  if(marks<0||marks>100||!Number.isFinite(marks)){toast("Marks must be from 0 to 100");return;}

  const passed=attPercentage(cid,a.batch)>=settings.attendanceThreshold&&marks>=settings.passMark;

  a.marks.push({candidate:cid,marks,outcome:passed?"Passed":"Not Passed"});

  if(passed&&!db.certificates.some(c=>c.candidate===cid))db.certificates.push({id:"CERT-"+Date.now().toString().slice(-6)+"-"+cid.slice(-3),candidate:cid,batch:a.batch,marks,issued:today(),status:"Certificate Generated"});

 }

 a.status="Marks Submitted";log(a,"Marks submitted and certificate rules applied");

 closeModal();render();toast("Marks saved");

}


/* OJT AND PLACEMENTS */

function renderSkillPlacement(){

 const certified=db.certificates.filter(c=>find("candidates",c.candidate));

 return page("OJT & Placements","KASE ERP › Skill Development › Placement",`

 <div class="card"><h3>Certified Candidate Post-Training Register</h3>

 <p class="muted">TSPs enter OJT and placement data through their portal.</p>

 <button class="btn primary" onclick="postTrainingDialog()">Preview TSP Post-Training Entry</button>

 ${tableHtml(["Candidate","Batch","Certificate","OJT Completion","Employer","Salary","Placement"],certified,c=>{const p=postFor(c.candidate);return[find("candidates",c.candidate)?.name,find("skillBatches",c.batch)?.na
```

```
me,c.id,p?.ojtDate||"Pending",p?.employer||"Pending",p?.salary?money(p.salary):"—
",p?.placementStatus||"Pending"]},c=>`openPostTraining('${c.candidate}')`)}

 </div>`);

}

function postTrainingDialog(){

 const rows=db.certificates.filter(c=>!postFor(c.candidate));

 if(!rows.length){toast("No certified candidates need post-training entry");return;}

 openModal("TSP Portal – Post-Training Entry",`

 <div class="field"><label>Certified Candidate</label><select
id="postCandidate">${rows.map(c=>`<option
value="${c.candidate}">${esc(find("candidates",c.candidate)?.name)} –
${esc(c.id)}</option>`).join("")}</select></div>

 <div class="field"><label>OJT Completion Date</label><input id="postOjt" type="date"></div>

 <div class="field"><label>Placement Status</label><select id="postStatus"><option value="Seeking
Placement">Seeking Placement</option><option value="Placed">Placed</option><option
value="Not Placed">Not Placed</option><option value="Offer Received">Offer
Received</option></select></div>

 <div class="field"><label>Company Name</label><input id="postEmployer"></div>

 <div class="field"><label>Monthly Salary (Rs.)</label><input id="postSalary" type="number"
min="0"></div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button
class="btn primary" onclick="savePostTraining()">Save</button></div>`);

}

function savePostTraining(){

 const
cid=$("postCandidate").value,status=$("postStatus").value,employer=$("postEmployer").value.trim()
;

 if(status==="Placed"&&!employer){toast("Company name required for placed candidates");return;}

 const p={id:"POST-"+Date.now().toString().slice(-
6),candidate:cid,ojtDate:$("postOjt").value,placementStatus:status,employer,salary:Number($("post
Salary").value||0),history:[]};

 db.postTraining.push(p);

 const
c=find("candidates",cid);c.placement=status;c.employer=employer;c.salary=p.salary;log(c,"Post-
training record updated");

 closeModal();render();toast("Post-training record saved");
```

```
}
function openPostTraining(id){
 const p=postFor(id),c=find("candidates",id);
 openModal("Post-Training – "+c?.name,`${p?`<div
class="formgrid">${fieldDisplay("Candidate",c?.name)}${fieldDisplay("OJT
Completion",p.ojtDate||"—")}${fieldDisplay("Placement
Status",p.placementStatus)}${fieldDisplay("Employer",p.employer||"—
")}${fieldDisplay("Salary",money(p.salary))}</div>`:"No post-training record."}<div
class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);
}

/* MILESTONES AND FICO */
function proposalMetrics(p){
 const cs=proposalCandidates(p.id).filter(c=>c.status!=="Dropout");
 const certs=db.certificates.filter(x=>cs.some(c=>c.id===x.candidate));
 const placed=certs.filter(x=>postFor(x.candidate)?.placementStatus==="Placed");
 return{
  target:Number(p.target||0),mobilized:cs.length,certified:certs.length,placed:placed.length,
  enrollment:p.target?cs.length/p.target*100:0,
  certification:cs.length?certs.length/cs.length*100:0,
  placement:certs.length?placed.length/certs.length*100:0
 };
}
function renderMilestones(){
 const rows=db.skillProposals.filter(p=>p.status==="Approved");
 return page("Milestone Payments / FICO","KASE ERP › Skill Development › Financial Triggers",`
 <div class="card"><h3>Training Milestone Register</h3>
 <p class="muted">Illustrative thresholds: 100% enrolment, 80% certification and 50% placement.
Confirm actual scheme rules before operational use.</p>

${tableHtml(["Proposal","TSP","Target","Mobilised","Certified","Placed","Enrolment","Certification",
"Placement"],rows,p=>{const
m=proposalMetrics(p);return[p.id,p.tsp,m.target,m.mobilized,m.certified,m.placed,m.enrollment>=
100?"Achieved":"Pending",m.certification>=80?"Achieved":"Pending",m.placement>=50?"Achieved
":"Pending"]},p=>`openMilestone('${p.id}')`)}
```

```
</div>

<div class="card"><h3>FICO Request Queue – Demo</h3>

${tableHtml(["Request","TSP","Milestone","Amount","Ledger","eOffice Reference","Status"],db.ficoQueue,q=>[q.id,q.tsp,q.milestone,money(q.amount),q.ledger||q.district,q.eoffice,q.status])}

</div>`);

}

function openMilestone(id){

const p=find("skillProposals",id),m=proposalMetrics(p);

const ms=[

{id:"Tranche 1",label:"Enrolment",percent:m.enrollment,threshold:100,count:m.mobilized+"/"+m.target,met:m.enrollment>=100},

{id:"Tranche 2",label:"Certification",percent:m.certification,threshold:80,count:m.certified+"/"+m.mobilized,met:m.certification>=80&&m.mobilized>0},

{id:"Tranche 3",label:"Placement",percent:m.placement,threshold:50,count:m.placed+"/"+m.certified,met:m.placement>=50&&m.certified>0}

];

openModal("Milestones – "+p.id,`

<div class="formgrid">${fieldDisplay("TSP",p.tsp)}${fieldDisplay("Scheme",schemeName(p.scheme))}${fieldDisplay("Target",m.target)}${fieldDisplay("Mobilised",m.mobilized)}${fieldDisplay("Certified",m.certified)}${fieldDisplay("Placed",m.placed)}</div>

${tableHtml(["Milestone","Numerator / Denominator","Progress","Threshold","Status"],ms,x=>[x.id,x.count,x.percent.toFixed(1)+"%",x.threshold+"%",x.met?"Achieved":"Pending"])}

<p class="muted">An approved eOffice reference and approved payment amount are mandatory.</p>

<div class="field"><label>Achieved Milestone</label><select id="payMilestone">${ms.filter(x=>x.met).map(x=>`<option value="${x.id}">${x.id} – ${x.label}</option>`).join("")}</select></div>

<div class="field"><label>Approved eOffice Proceeding *</label><input id="payEoffice"></div>

<div class="field"><label>Approved Payment Amount (Rs.) *</label><input id="payAmount" type="number" min="1"></div>
```

```
 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="submitFicoRequest('${p.id}')">Generate Report & Queue Request</button></div>`);

 if(!$("payMilestone").options.length)$("payMilestone").innerHTML='<option value="">No milestone achieved</option>';

}

function submitFicoRequest(id){

 const p=find("skillProposals",id),milestone=$("payMilestone").value,eoffice=$("payEoffice").value.trim(),amount=Number($("payAmount").value);

 if(!milestone){toast("No achieved milestone");return;}

 if(!eoffice||amount<=0){toast("Approved eOffice reference and amount required");return;}

 if(db.ficoQueue.some(q=>q.proposal===id&&q.milestone===milestone)){toast("Request already exists for this milestone");return;}

 const s=find("skillSchemes",p.scheme);

 if(p.stream==="State"&&!p.ledger&&!s?.ledger){toast("Map the State scheme to a budget sub-ledger first");return;}

 db.ficoQueue.unshift({id:"FICO-SD-"+Date.now().toString().slice(-6),proposal:id,scheme:p.scheme,tsp:p.tsp,milestone,amount,eoffice,ledger:p.ledger||s?.ledger||"",district:p.district,status:"Ready for AP Workflow",history:[]});

 log(p,"Milestone request queued with eOffice reference "+eoffice);

 closeModal();render();toast("Simulated FICO request queued; no live transmission occurred");

}


/* STATE BUDGET LEDGER MAPPING */

function renderStateLedgers(){

 const rows=db.skillSchemes.filter(s=>s.stream==="State");

 return page("State Scheme Budget Mapping","KASE ERP › Skill Development › State Funds",`

 <div class="card"><h3>Scheme-to-FICO Sub-Ledger Mapping</h3>

 ${tableHtml(["Scheme","Funding Source","Mapped Ledger","Ledger Name","Allocated Budget","Status"],rows,s=>[s.name,s.fundSource,s.ledger||"Not mapped",db.stateLedgers.find(l=>l.id===s.ledger)?.name||"—",money(db.stateLedgers.find(l=>l.id===s.ledger)?.allocated||0),s.ledger?"Mapped":"Mapping Required"],s=>`mapStateLedger('${s.id}')`)}

 </div>

 <div class="card"><h3>Available Sub-Ledgers</h3>
```

```
${tableHtml(["Ledger","Name","Allocated","Utilised","Balance"],db.stateLedgers,l=>[l.id,l.name,money(l.allocated),money(l.utilized),money(l.allocated-l.utilized)])}

 </div>`);

}

function mapStateLedger(id){

 if(!can("stateBudget")&&!can("all")){toast("Your role cannot change ledger mappings");return;}

 const s=find("skillSchemes",id);

 openModal("Map Ledger – "+s.name,`

 <div class="field"><label>Budget Ledger</label><select id="ledgerChoice">${db.stateLedgers.map(l=>`<option value="${l.id}" ${s.ledger===l.id?"selected":""}>${esc(l.id)} – ${esc(l.name)}</option>`).join("")}</select></div>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Cancel</button><button class="btn primary" onclick="saveStateLedger('${id}')">Save Mapping</button></div>`);

}

function saveStateLedger(id){

 const s=find("skillSchemes",id);s.ledger=$("ledgerChoice").value;log(s,"Budget ledger mapped to "+s.ledger);

 db.skillProposals.filter(p=>p.scheme===id).forEach(p=>p.ledger=s.ledger);

 closeModal();render();toast("Ledger mapping saved");

}


/* DSDC BUDGETS */

function renderDSDC(){

 const rows=db.dsdcBudgets;

 const allocated=rows.reduce((sum,x)=>sum+Number(x.allocated||0),0);

 const utilized=rows.reduce((sum,x)=>sum+Number(x.utilized||0),0);

 const balance=allocated-utilized;

 return page("DSDC Operations & Fund Utilisation","KASE ERP › Skill Development › DSDC",`

 <div class="card"><h3>District Skill Development Centre Budgets</h3>

 <p class="muted">Utilisation is monitored against the designated district allocation and ledger.</p>

 <div class="cards">

 ${metricCard("Districts",rows.length)}
```

```
 ${metricCard("Total Allocation",money(allocated))}

 ${metricCard("Total Utilised",money(utilized))}

 ${metricCard("Balance",money(balance))}

 </div>

 ${tableHtml(["Budget Ref.","District","Budget Source","FICO
Ledger","Allocated","Utilised","Balance","Officer"],rows,r=>[r.id,r.district,r.source,r.ledger,money(r.
allocated),money(r.utilized),money(r.allocated-r.utilized),r.officer],r=>`openDistrict('${r.id}')`)}

 </div>

 <div class="card"><h3>DSDC Training Batches</h3>

 ${tableHtml(["Batch","District","Provider","Candidates","Status","Budget
Ledger"],db.skillBatches.filter(b=>b.budgetId),b=>[b.name,b.district,proposalName(b.proposal),b.ca
ndidateIds.length,b.status,b.budgetId],b=>`openBatch('${b.id}')`)}

 </div>`);

}

function openDistrict(id){

 const d=find("dsdcBudgets",id),rows=db.skillBatches.filter(b=>b.budgetId===id);

 openModal("District Budget – "+d.district,`

 <div class="formgrid">${fieldDisplay("District",d.district)}${fieldDisplay("Budget Reference",d.id)}

 ${fieldDisplay("Source",d.source)}${fieldDisplay("FICO Ledger",d.ledger)}

 ${fieldDisplay("Allocated",money(d.allocated))}${fieldDisplay("Utilised",money(d.utilized))}

 ${fieldDisplay("Available Balance",money(d.allocated-
d.utilized))}${fieldDisplay("Officer",d.officer)}</div>

 <h3>Associated
Batches</h3>${tableHtml(["Batch","Candidates","Status"],rows,b=>[b.name,b.candidateIds.length,b
.status])}

 <p class="muted">District funds must not be cross-utilised.</p>

 <div class="modalactions"><button class="btn" onclick="closeModal()">Close</button></div>`);

}


/* REPORTS */

function renderReports(){

 return page("Reports & Audit","KASE ERP › Skill Development › Reports",`

 <div class="card"><h3>Available Reports</h3><div class="cards">
```

```
 ${metricCard("Skill Scheme Execution","Generate","generateReport('skillProposals','Skill Scheme Execution')")}

 ${metricCard("Batch & Attendance","Generate","generateReport('skillBatches','Batch and Attendance')")}

 ${metricCard("Certification & Placement","Generate","generateReport('certificates','Certification and Placement')")}

 ${metricCard("State Fund Utilisation","Generate","generateReport('stateLedgers','State Fund Utilisation')")}

 ${metricCard("DSDC Fund Utilisation","Generate","generateReport('dsdcBudgets','DSDC Fund Utilisation')")}

 ${metricCard("FICO Requests","Generate","generateReport('ficoQueue','FICO Request Queue')")}

 </div></div>

 <div class="card"><h3>Generated Reports</h3>

${tableHtml(["Reference","Report","Created","Status"],db.reports,r=>[r.id,r.title,r.created,r.status])}

 </div>`);

}

function generateReport(m,title){

 const rows=db[m]||[];

 db.reports.unshift({id:"RPT-"+Date.now().toString().slice(-6),title,type:m,created:today(),status:"Generated"});

 toast(title+" report generated");

 if(rows.length){

  const keys=[...new Set(rows.flatMap(r=>Object.keys(r).filter(k=>typeof r[k]!=="object")))];

  exportCSV("KASE_"+m+"_Report.csv",[keys,...rows.map(r=>keys.map(k=>r[k]??""))]);

 }

 render();

}

/* GLOBAL SEARCH */

$("globalSearch").addEventListener("keydown",e=>{

 if(e.key!=="Enter")return;

 const q=e.target.value.toLowerCase().trim();if(!q)return;
```

```
 const m=Object.entries(moduleNames).find(([id,label])=>label.toLowerCase().includes(q));
 if(m){openModule(m[0]);return;}
 const
found=Object.entries(db).find(([id,rows])=>rows.some(r=>JSON.stringify(r).toLowerCase().includes(q
)));
 const
mapped={skillSchemes:"skillSchemes",skillProposals:"skillProposals",trainers:"skillTrainers",candidat
es:"skillCandidates",skillBatches:"skillBatches",attendanceLogs:"skillAttendance",assessments:"skillA
ssessment",certificates:"skillAssessment",postTraining:"skillPlacement",dsdcBudgets:"dsdc",stateLe
dgers:"stateLedgers",ficoQueue:"skillMilestones",reports:"reports"};
 if(found&&mapped[found[0]]){openModule(mapped[found[0]]);return;}
 toast("No matching Skill Development module or record found");
});

$("dateLabel").textContent=new Date().toLocaleDateString("en-GB",{day:"2-
digit",month:"short",year:"numeric"});
document.body.classList.add("login-mode");
render();
loginDialog();
</script>
</body>
</html>
```